

What even is IMU?

The Blindfolded Robot: The Definitive Guide to Inertial Navigation

The Concept

Imagine you are kidnapped, blindfolded, and thrown into the passenger seat of a car. You cannot see the road, hear the engine, or check a GPS.

Can you draw a map of exactly where the car went just by "feeling" the motion?

This is exactly what an **IMU (Inertial Measurement Unit)** does. It uses physics equations to turn "feelings" (Forces/Rotation) into a "Map" (Position).

Part 1: The Physics Hierarchy (The Vocabulary)

To understand navigation, you must understand that motion happens in "Levels." To go **UP** the ladder (to find position), we accumulate data over time (Integration). To go **DOWN**, we measure rates of change (Derivation).

Level 1: Position (p)

- **Definition:** Where you are on the map.
- **Analogy:** A pin dropped on Google Maps.
- **Unit:** Meters (m).

Level 2: Velocity (v)

- **Definition:** How fast your **Position** is changing.
- **Analogy:** The speedometer needle.
- **Unit:** Meters per second (m/s).

Level 3: Acceleration (a)

- **Definition:** How fast your **Velocity** is changing.
- **Analogy:** The "Push" you feel in your stomach when the driver hits the gas pedal.
- **Unit:** Meters per second squared (m/s^2).
- **Sensor:** The IMU **Accelerometer** lives here.

Level 4: Jerk (j)

- **Definition:** How fast your **Acceleration** is changing.
 - **Analogy:** The "Smoothness" of the driver.
 - **High Jerk:** The driver stomps on the gas instantly. (Snap!)
 - **Low Jerk:** The driver slowly presses the pedal down. (Smooth).
 - **Unit:** Meters per second cubed (m/s^3).
-

Part 2: The Math of Integration (Deriving the Formulas)

You will often see weird numbers like 0.5 or 1/6 in navigation code. Here is the calculus derivation of where they come from.

The Goal: Find Distance (Position) based on Motion.

Formula A: The Push (Acceleration → Position)

If you accelerate constantly, distance is the area under the velocity graph (a triangle).

- **Calculus Rule:** $\int t \, dt = \frac{1}{2}t^2$.
- **The Math:**

$$\text{Distance}_A = \frac{1}{2} \times \text{Acceleration} \times \text{Time}^2$$

Formula B: The Smooth Ramp (Jerk → Position)

Real robots don't accelerate instantly; they ramp up. This ramp is a curve (Parabola).

- We need to integrate Jerk three times to get to Position.
 1. Jerk → Accel: $\int 1 \, dt = t$
 2. Accel → Vel: $\int t \, dt = \frac{1}{2}t^2$
 3. Vel → Pos: $\int \frac{1}{2}t^2 \, dt = \frac{1}{2} \times \frac{t^3}{3} = \frac{1}{6}t^3$
- **The Math:**

$$\text{Distance}_B = \frac{1}{6} \times \text{Jerk} \times \text{Time}^3$$

✓ The Master Equation

The code adds up every factor to find the exact new position:

$$P_{new} = P_{old} + (V \cdot t) + (0.5 \cdot a \cdot t^2) + \left(\frac{1}{6} \cdot j \cdot t^3\right)$$

Part 3: The Hardware (The Sensors)

The IMU is a tiny chip containing two specific sensors acting as your inner ear.

1. The Accelerometer (The Linear Sensor)

It measures **Linear Acceleration** (Level 3).

- **What it feels:** Pushes in X (Forward), Y (Left), and Z (Up).
- **The "Earth Tax":** Even when the robot is parked, the accelerometer reads **9.8 m/s^2** Upwards.
 - *Why?* Because gravity pulls the sensor down into the floor, and the floor pushes back up. The sensor measures that "Normal Force" from the floor.

2. The Gyroscope (The Rotation Sensor)

It measures **Angular Velocity** (ω or Omega).

- **What it feels:** How fast is the robot spinning?
 - **Analogy:** Spinning in an office chair with your eyes closed. You can feel *how fast* you are spinning, but you don't know *which direction* you are facing (North vs. South).
 - **Unit:** Radians per second.
-

Part 4: The Algorithm (The "Black Box" Workflow)

How does the robot actually process the data? It follows a precise pipeline.

Step 1: Calibration (Finding "Zero")

The Problem: Sensors are imperfect.

- **Gyro:** When still, it says "Spinning at 0.01 rad/s."
- **Accel:** When still, it says "Accelerating at 9.81 m/s^2 ."

The Solution (`execute_imu_calibration`):

1. **Gyro Bias:** Average the first 100 samples while stopped. Subtract this average from all future readings.
2. **Accel Bias:**
 - Calculate the average vector: $\vec{v}_{avg} = [0.1, 0.2, 9.9]$.
 - Identify "Down": Since the robot is stopped, $\vec{v}_{avg}$ is the direction of gravity.
 - Separate Bias: $\text{Bias} = \vec{v}_{avg} - \text{GravityVector}$.

Step 2: Back-Propagation (The "Rewind")

The Problem: The LiDAR took a picture at 12:00:03 . The IMU wrote its note at 12:00:00 .

The Solution:

- We know the state of the robot *now* (at 03).
- We use the physics formulas in **Reverse** (using negative time) to calculate exactly where the robot was 3 seconds ago (at 00).
- **Why?** This aligns the "Starting Line" of the math with the IMU data stream so we can integrate forward smoothly.

Step 3: Quaternion Integration (The Magic Compass)

We need to track orientation using **Quaternions** (q) to avoid math errors (Gimbal Lock).

- **The Code:** `q_next = q + 0.5 * q * omega * dt`
- **The Magic Explained:**
 1. **Physics:** The rate of change of a quaternion is $\dot{q} = \frac{1}{2}\vec{\omega} \otimes q$.
 2. **Why 0.5?** Quaternions rotate vectors using "double cover" algebra (qvq^{-1}). Algebraically, they rotate "twice as fast" as the vector they represent. The 0.5 slows the math down to match physical reality.
 3. **Integration:** We add this rate of change to the old orientation to find the new direction.

Step 4: Transformation (Seat vs. World)

This is the most critical concept.

- **Body Frame (Seat):** The IMU moves with the car. If the car tilts up a hill, the IMU feels gravity pulling it **Backwards**.
- **World Frame (Map):** Gravity always pulls **Straight Down**.
- **The Action:** The code rotates the "Felt Forces" by the "Magic Compass" (q) to see which way they point on the Map.

Step 5: Deleting the Earth (Gravity Removal)

Once rotated to the Map frame, the robot says:

"I see a force of 9.8 pulling down. Is that the engine?"

"No, that's just Earth."

- **Action:** Subtract 9.8 from the Z-axis.
- **Result:** Whatever force remains is the **True Motion** of the robot.

Part 5: The Parameters

1. What is the `smatrix` ? (Scale & Misalignment)

The `smatrix` (Scale Matrix) is a 3×3 correction grid that fixes two specific physical defects in the sensor chip.

Think of your IMU chip. Inside, there are three tiny sensors (springs/capacitors) meant to measure X, Y, and Z.

Defect A: Scale Error (The Stretched Ruler)

- **Ideal:** If you apply 1G of force, the sensor reads `1.00`.
- **Reality:** The sensor might be slightly too sensitive and read `1.01`.
- **The Fix:** We need to multiply the reading by `0.99` to shrink it back to reality.
- **Where in Matrix:** These are the **Diagonal** numbers (S_{xx}, S_{yy}, S_{zz}).

Defect B: Misalignment / Cross-Axis (The Crushed Box)

- **Ideal:** The X, Y, and Z sensors should be perfectly 90 degrees apart (Orthogonal).
- **Reality:** Manufacturing isn't perfect. The X-axis might be 89.9 degrees from the Y-axis.
- **Consequence:** When you push **Purely Forward (X)**, the crooked Y-sensor feels a tiny bit of that push.
- **The Fix:** We use the off-diagonal numbers to "subtract" the X-force leaking into the Y-sensor.
- **Where in Matrix:** These are the **Off-Diagonal** numbers (S_{xy}, S_{yx} , etc.).

The Math

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}_{corrected} = \begin{bmatrix} \mathbf{S}_{xx} & S_{xy} & S_{xz} \\ S_{yx} & \mathbf{S}_{yy} & S_{yz} \\ S_{zx} & S_{zy} & \mathbf{S}_{zz} \end{bmatrix} \times \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}_{raw}$$

Where do I get the `smatrix`?

Usually, you don't calculate this yourself every run. This is a Factory Calibration.

- **High-end IMUs:** Provide this on a datasheet.
- **Low-end IMUs:** You assume it is the Identity Matrix (perfect) unless you perform a rigorous "6-Point Tumble Test" in a lab.

2. How to Measure Bias (The Practical Guide)

Unlike the `smatrix` (which stays the same for years), **Bias changes every time you turn on the robot** (and shifts with temperature). You must measure it **Online** (every time the code starts).

Here are the two ways to do it.

Method A: The "Just Sit Still" Method

This is what your `execute_imu_calibration` function is doing.

The Procedure:

1. **Power on the robot.**
2. **DO NOT TOUCH IT.** Ideally, leave it parked on flat ground, but a hill is okay too
3. **Wait:** Let the code collect data for ~2-5 seconds (e.g., 200-500 samples).

The Calculation (Internal):

1. **Gyro Bias:** The robot is not spinning.
 - `Bias_Gyro = Average(All Gyro Samples)`
2. **Accel Bias:** The robot is not accelerating (only Gravity).
 - `Average_Vector = Average(All Accel Samples)`
 - `Gravity_Vector = Average_Vector` (assuming `smatrix` is already applied or ignored).
 - `Bias_Accel` is technically `Average_Vector - Expected_Gravity`, but since we define the average vector as gravity during alignment, the bias is often absorbed or set to zero relative to that initial pose.

⚠ **Crucial Requirement** If you nudge, bump, or drive the robot during those first 2 seconds, the calibration will fail. The robot will think "Turning Left" is actually "Stopped." You will drift forever.

Method B: The "Allan Variance" (Lab / Deep Analysis)

If you want to know *how good* your sensor is (e.g., "Does this \$10 IMU drift more than this \$1000 IMU?"), you perform an Allan Variance test.

1. Put the IMU in a basement (temperature stable, no vibration).
2. Record data for **12 Hours**.
3. Plot the noise.
4. This doesn't give you the bias for *today*, but it tells you how much the bias will *change* (drift) over 10 minutes.

Summary Strategy for You

1. **For `smatrix`:** Assume it is Identity (perfect) unless you have a factory datasheet.
 2. **For Bias:** Ensure your startup procedure includes a "**Hands Off**" period of 3-5 seconds so the `execute_imu_calibration` function can capture a clean average.
-

Part 5: The "Drift" Trap (Why we need Damping)

If the math is perfect, why do robots get lost without GPS?

The Snowball Effect

Because we calculate Position based on Velocity, and Velocity based on Acceleration... a tiny error at the bottom **snowballs** to the top.

1. **Sensor Error:** "I think we accelerated 0.00001% more than we did."
2. **Velocity:** "Okay, I will add that speed to our total *forever*."
3. **Position:** "Since we have that phantom speed, we are drifting miles away."

The Solution: Damping (Virtual Friction)

We cheat. We tell the math that there is "Air Resistance."

- **Code:** `velocity = velocity * 0.99`
 - **Why:** This slowly kills the "phantom speed" errors over time. If the robot stops accelerating, this math ensures the velocity eventually returns to zero, preventing the robot from "drifting" to infinity on the map.
-

Part 6: Practical Realities

1. Can IMU estimate Height? (Z-axis)

Short Answer: No.

Long Answer: Height is the double-integration of acceleration. A 0.1% sensor error in gravity removal results in an **18-meter error** after just 60 seconds. You cannot use IMU for hills without external correction (SLAM/Odometry).

2. State Persistence (Memory)

- **Position:** We reset it to 0 every scan. (We only care about local deskewing).
 - **Velocity:** We **KEEP** it. (If we are driving 60mph, we need to remember that momentum for the next scan, or the deskewing will fail).
 - **Bias:** We **KEEP** it. (Sensor errors change slowly; we keep the best estimate).
-

Summary Cheat Sheet

Term	Symbol	Analogy	Measurement Unit
Position	p	Pin on a map	Meters (m)

Term	Symbol	Analogy	Measurement Unit
Velocity	v	Speedometer	Meters/sec (m/s)
Acceleration	a	The Push	m/s^2
Jerk	j	Smoothness	m/s^3
Quaternion	q	Magic Compass	(4 numbers)
Bias	b	The "Tiny Lie"	varies
Integration	$\int$	Accumulating the past	N/A